Universitate Politehnica Timișoara
Facultate de Automatică și Calculatoare
Disciplina: Proiectarea cu Microprocesoare

Proiectarea unui microsistem cu microprocesorul 8086

Student: Șămanțu Astrid-Noémi

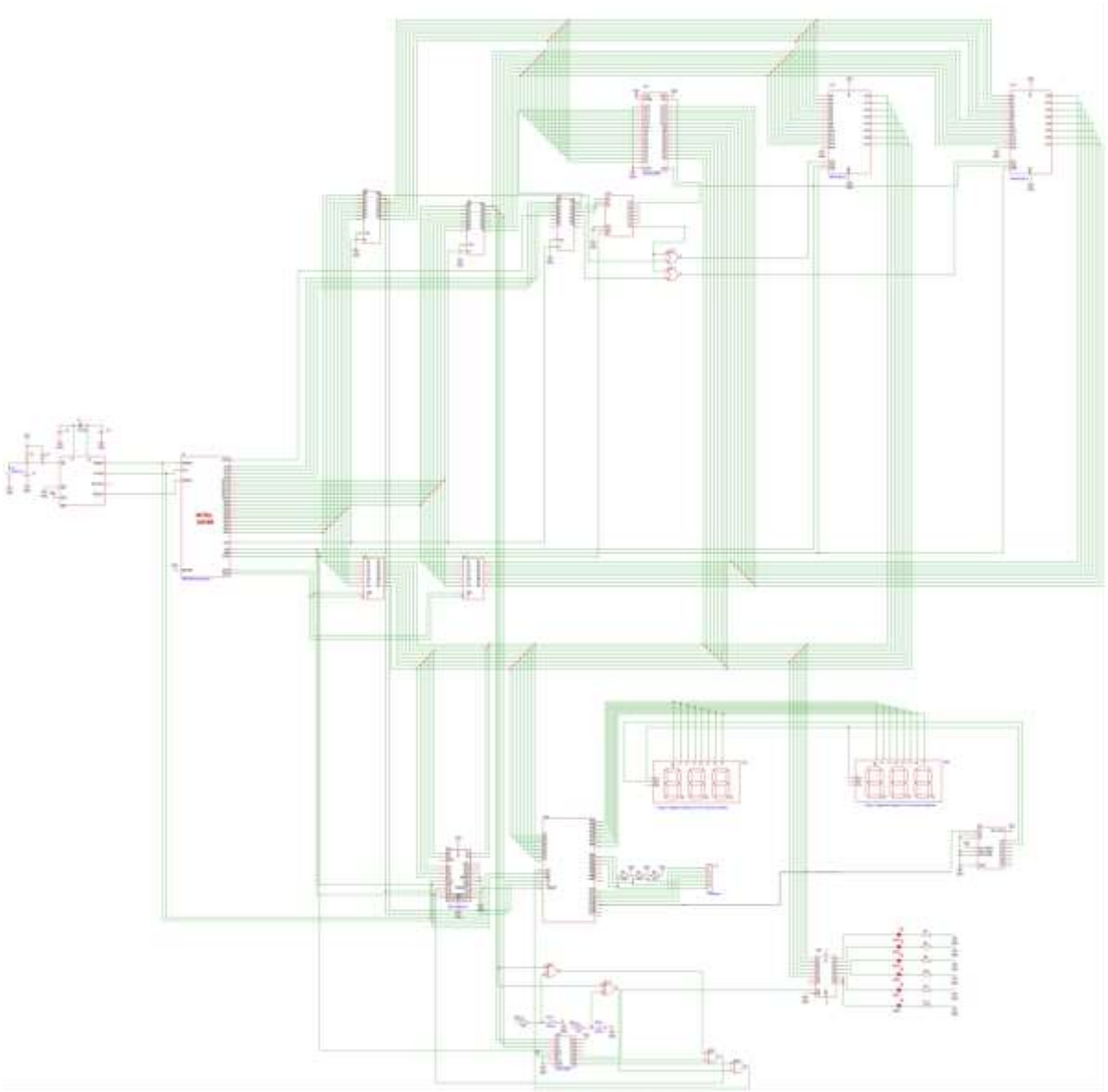
An universitar: 2025-2026

Tema Proiectului

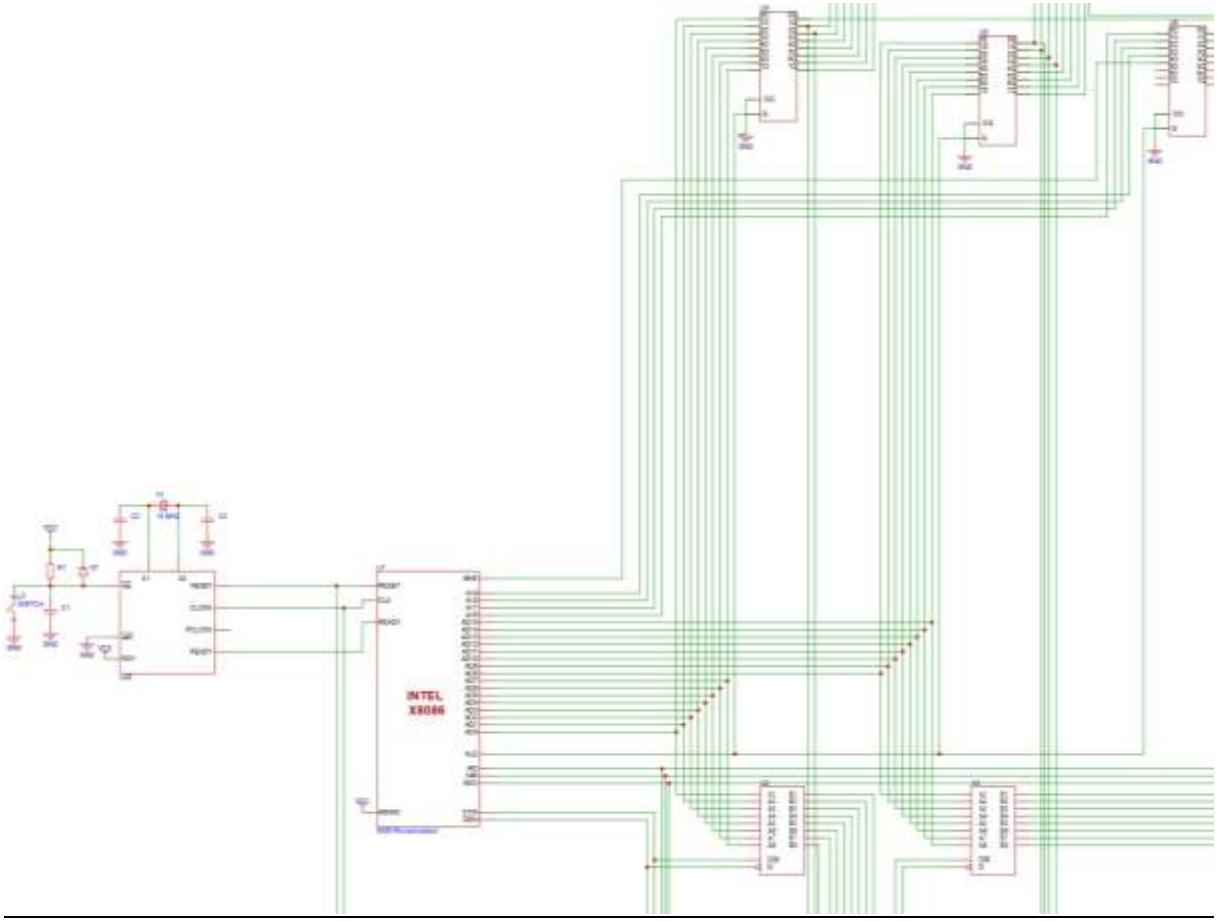
Să se proiecteze un microsistem cu următoarea structură:

- unitate centrală cu microprocesorul 8086;
- 128 KB memorie EPROM, utilizând circuite 27C2048;
- 64 KB memorie SRAM, utilizând circuite 62512;
- interfață serială, cu circuitul 8251, plasată în zona 0AF0H – - 0AF2H sau 0BF0H – 0BF2H, în funcție de poziția - microcomutatorului S1;
- interfață paralelă, cu circuitul 8255, plasată în zona 0D70H – - 0D76H sau 0C70H – 0C76H, în funcție de poziția - microcomutatorului S2;
- o minitastatură cu 12 contacte;
- 6 led-uri;
- un modul de afișare cu 7 segmente, cu 6 ranguri;

Descrierea hardware-ului



Unitatea Centrală



Microprocesorul Intel 8086

Microprocesorul este elementul central al sistemului. Este un procesor cu arhitectura pe 16 biți. Acesta are o magistrală de date internă și externă de 16 biți și o magistrală de adrese de 20 de biți.Procesorul permite adresarea unui spațiu fizic de memorie de 1 MB**.** Este configurat să funcționeze în modul minim (Min Mode) prin conectarea pinului MN/MX la VCC(+5V). onfigurația permite procesorului să genereze singur semnalele de comandă pentru magistrală (de exemplu !RD, !WR, M/!IO), fără afi nevoiede un controller de magistrală extern.

Generatorul de Ceas 8284A

Generatorul de Ceas asigură sincronizarea întregului sistem. Se conectează la un cristal de cuarț de 15 MHz și furnizează procesorului un semnal de tact (CLK) de 5 MHz (cedivizează frecvența cristalului cu 3), semnalul RESET șisemnalul READY.

Sistemul de demultiplexare (Latch-urile 74x373)

Procesorul 8086 utilizează o magistrală multiplexată timp (liniile AD0-AD15 transportă alternativ adrese și date),așada separarea informației de adresă este necesară.. Am utilizat trei circuite 74x373 (latch-uri) controlate de semnalul ALE (Address Latch Enable) care este generat de procesor:

- Funcționare: Pe frontul descrescător al semnalului ALE, latch-urile memorează adresa prezentă pe magistrala multiplexată.

- Rezultat: Se obține o magistrală de adrese stabilă (A0-A19) și semnalul !BHE valid pe toată durata ciclului mașină, astfel perimițând memoriei și perifericelor să fie adresate corect chiar și atunci când liniile AD ale procesorului trec la transferul de date.

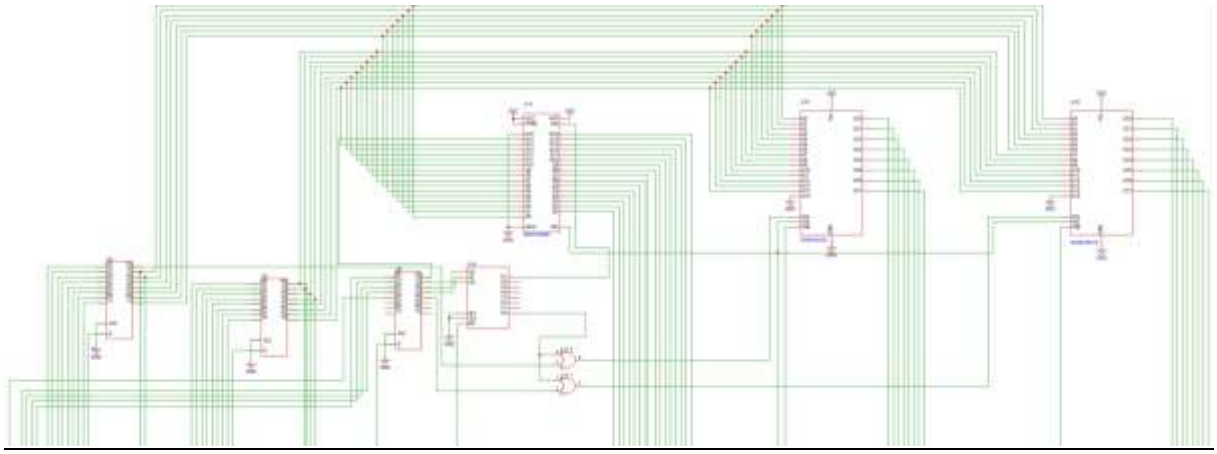
Interfața cu magistrala de date (Amplificatoarele 74x245)

Pentru a asigura izolarea procesorului și capacitatea de curent necesară pentru a comanda toate cipurile de memorie și I/O, am utilizat două circuite 74x245. Aceste circuite conectează magistrala locală a procesorului (AD0-AD15) la magistrala de date a sistemului (D0-D15). Funcționarea circuitelor este controlată strict de procesor:

- Direcția (DT/!R): Stabilește în ce sens circulă datele, dinspre procesor spre sistem (Scriere) sau invers (Citire).

- Validarea (!DEN): Doar atunci când datele sunt valide pe magistral acesta activează amplificatoarele, evitând conflictele electrice (bus contention).

Sistemul de Memorie



Organizarea Memoriei

Microprocesorul 8086 utilizând o magistrală de date de 16 biți, subsistemul de memorie a fost proiectat pentru a permite transferuri de date pe cuvânt (16 biți) sau pe octet (8 biți). Spațiul de adresare este împărțit fizic în două bancuri pentru memoria RAM. Memoria ROM este accesată direct pe magistrala completă de 16 biți.

Memoria Program (EPROM)

Pentru a stoca codul programului, am utilizat un circuit de memorie de tip 27C2048 (organizare 128K x 16 biți).

Capacitate și Amplasare: Deși capacitatea fizică a cipului este de 256 KB, am limitat utilizarea acestuia la 128 KB prin conectarea liniei de adresă A16 a cipului la masă (GND).

Conectare: Liniile de date Q0-Q15 sunt conectate direct la magistrala de date a sistemului (D0-D15, fiind o memorie organizată pe 16 biț,. Selecția cipului este realizată prin semnalul !CS care este generat de decodificatorul de adrese.

Memoria de Date (SRAM)

Pentru datele volatile și stivă, am alocat 64 KB de memorie SRAM, ce a fost plasată la baza spațiului de memorie. Implementarea utilizează două circuite 62512 (organizare 64K x 8 biți), configurate în paralel:

Bancul Par (Low): Conectat la liniile de date D0-D7, selectat atunci când procesorul accesează o adresă pară (A0 = 0).

Bancul Impar (High): Conectat la liniile de date D8-D15, selectat prin semnalul !BHE (Bus High Enable).

Logica de Selecție: Având în vedere că scrierea poate avea loc pe un singur octet, semnalele de selecție a cipurilor (!CS) sunt generate prin porți logice SAU (74x32). Acestea combină semnalul de selecție globa, de la decodificator, cu semnalele A0 și !BHE ale procesorului, asigurând activarea individuală sau simultană a bancurilor.

Decodificarea Adreselor

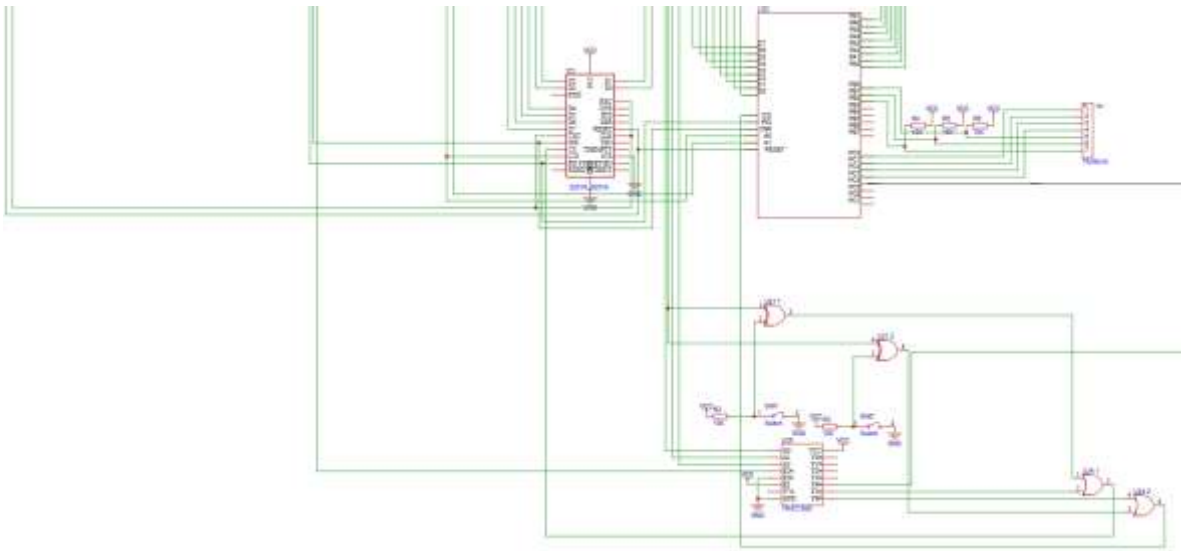
Selecția zonelor de memorie este realizată de un decodificator 74x138, care monitorizează liniile de adresă cele mai semnificative (A17, A18, A19).

Ieșirea !Y0: Activează memoria SRAM în intervalul 00000H - 1FFFFH (utilizăm doar primii 64KB).

Ieșirea !Y7: Activează memoria EPROM în intervalul E0000H - FFFFFH.

Alinierea Adreselor: Deoarece procesorul folosește magistrala multiplexată și semnale de banc, pinii de adresă ai memoriilor (A0...An) sunt conectați decalat la magistrala sistemului (A1...An+1).

Sistemul de Intrări/Ieșiri



Pentru comunicarea cu exteriorul, sistemul utilizează o arhitectură de tip I/O Mapped (Intrări/Ieșiri mapate izolat), ce separă spațiul de adrese al memoriei de cel al perifericelor. Deoarece circuitele de interfață utilizate (8251 și 8255) au o magistrală de date de 8 biți, ele sunt conectate la magistrala de date inferioară a sistemului (D0-D7). Această configurație impune faptul ca toate perifericele să fie accesate la adrese pare.

Logica de Selecție și Mapare Dinamică

Pentru a îndeplini cerința de modificare a adreselor de bază prin intermediul microcomutatoarelor (S1 și S2), am proiectat un etaj de decodificare compus din două niveluri:

Decodificarea Primară (74HC138): Un decodificator dedicat spațiului I/O (activat de semnalul M/!IO=0) ce monitorizează liniile de adresă intermediare A11, A10, A9. Acesta identifică zona generală de adresare: ieșirea !Y5 (cod 101) pentru interfața serială și ieșirea !Y6 (cod 110) pentru cea paralelă.

Selecția Finală (Logica XOR-OR): Semnalul final de selecție a cipului (!CS) este generat prin validarea bitului de adresă A8 în funcție de starea switch-urilor. Am utilizat porți XOR pentru a compara starea bitului A8 cu cea a switch-ului, iar rezultatul condiționează activarea cipului.

Interfața Serială (Intel 8251A)

Maparea Adreselor: Circuitul este mapat dinamic la adresa de bază 0AF0H (când S1 este închis/0) sau 0BF0H (când S1 este deschis/1).

Conexiuni de Control: Selecția internă între registrul de date și cel de comandă/stare (pinul C/!D) este conectată la linia de adresă A1 a sistemului. Astfel, datele sunt accesate la adresa de bază (ex: 0AF0H), iar comenzile la adresa de bază + 2 (ex: 0AF2H).

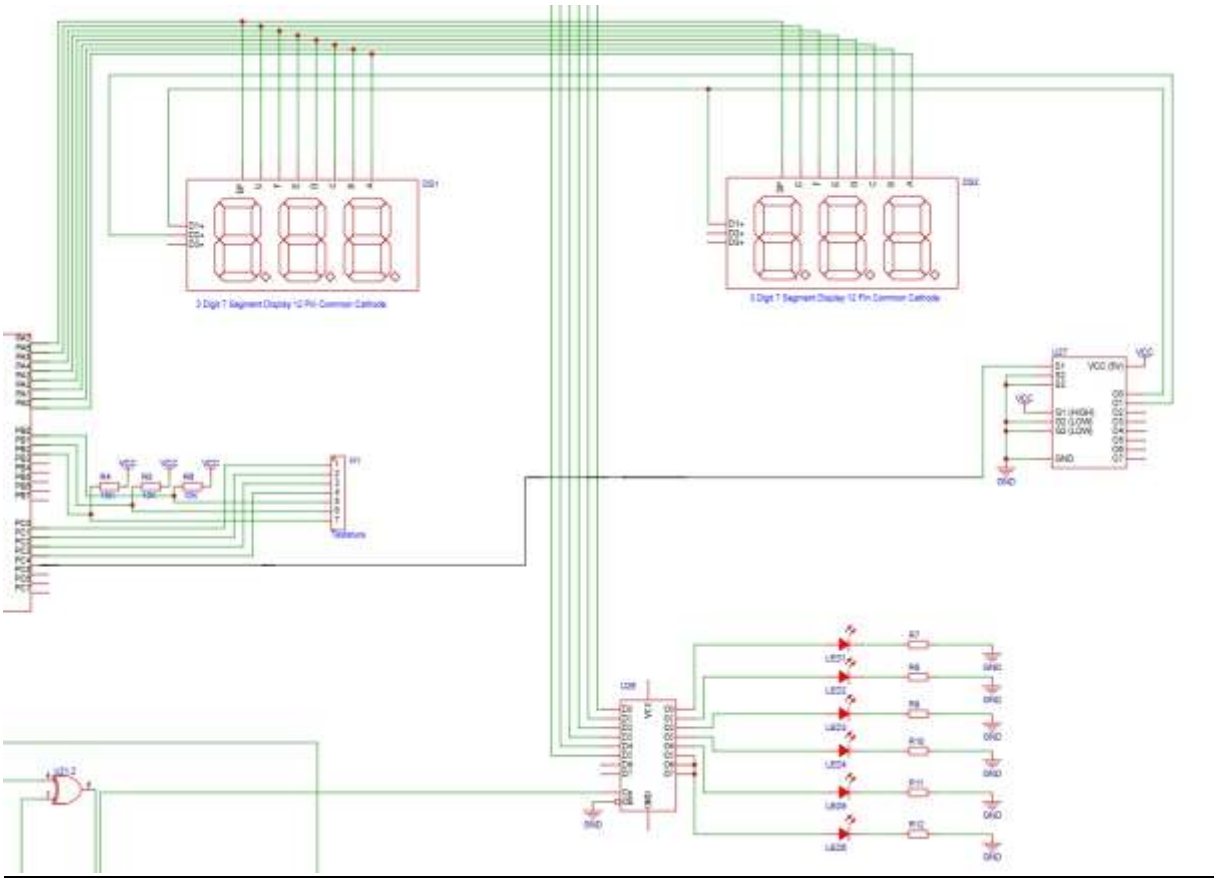
Sincronizare: Semnalele de citire (!RD) și scriere (!WR) sunt preluate direct de la unitatea centrală.

Interfața Paralelă (Intel 8255A)

Maparea Adreselor: Circuitul răspunde la adresa de bază 0D70H (când S2 este închis/0) sau 0C70H (când S2 este deschis/1).

Conexiuni de Control: Pentru a accesa cele 4 registre interne (Port A, Port B, Port C, Registru Control), liniile de selecție A0 și A1 ale cipului sunt conectate decalat la liniile A1 și A2 ale sistemului. Aceasta asigură o mapare la adrese pare consecutive (ex: 0D70H, 0D72H, 0D74H, 0D76H).

Configurarea și Conectarea Perifericelor



Pentru ca intrările și ieșirile disponibile să fie gestionate eficient, am configurat porturile circuitului 8255 în modul mixt (Cuvânt de control 81H), alocând funcții specifice fiecărui grup de pini.

Subsistemul de Afișare (Multiplexat)

Configurație: Portul A (PA0-PA7) este configurat ca IEȘIRE și este conectat la magistrala de date a segmentelor. Selecția afișajului activ se face prin pinul PC4 (Portul C Upper), configurat tot ca IEȘIRE.

Funcționare (Multiplexare): Comanda afișajelor se face dinamic, acestea fiind de tip Catod Comun. Pinul PC4 controlează un decodificator 74138:

- Starea 0 pe PC4 activează primul afișaj (DS1).
- Starea 1 pe PC4 activează al doilea afișaj (DS2).
- Această alternanță rapidă permite utilizarea aceloraşi linii de date (Port A) pentru ambele grupuri de cifre.

Interfața cu Tastatura Matriceală

Configurație: Pentru a permite scanarea tastatura (4 linii x 3 coloane) este divizată între două porturi. Liniile sunt pe Portul C (PC0-PC3 - Ieșire), iar coloanele pe Portul B (PB0-PB2 - Intrare).

Scanarea: Procesorul aplică secvențial un nivel "0" pe rânduri și citește starea coloanelor. Pentru a preveni citirile false rezistențele de pull-up (R4-R6) asigură un nivel logic "1" stabil pe coloane în lipsa apăsării.

Subsistemul de Semnalizare (LED-uri)

Conectare Independentă: Cele 6 LED-uri nu sunt conectate la circuitul 8255.Acestea sunt conectate direct la magistrala de date a sistemului (D0-D5), prin intermediul unui circuit Latch Octal (U26).

Rolul Latch-ului: Având în vedere că datele pe magistrală sunt volatile, latch-ul are rolul de a memora comanda de aprindere/stingere în momentul scrierii la adresa dedicată, menținând starea LED-urilor constantă fără a ocupa procesorul.

Programele, cu delimitări şi comentarii

8251

Rutinele de programare:

```
MOV AL,0CEH ; cuvânt de mod
OUT DX,AL
MOV AL,15H ; cuvânt de comandă
OUT DX,AL
RET
```

Rutinele de emisie/recepție caracter pe interfața serială:

```
IN AL, DX ; citire si testare TxRDY
RCR AL, 1
JNC TR
MOV AL, CL  ; se preia data din registrul CL
SUB DX, 1 ; Calculeaza adresa port Date (Status-1)
OUT DX, AL  ; Trimite caracterul
RET

IN AL, DX ; citire si testare RxRDY
RCR AL, 2
JNC REC
SUB DX, 1 ; Calculeaza adresa port Date (Status-1)
IN AL, DX ; se preia data de la 8251
MOV CL, AL ; se depune data in registrul CL
RET
```

8255

Rutina de programare:

```
MOV AL,81H
OUT DX,AL
RET
```

Rutina de emisie caracter pe interfață paralelă:

```
IN AL, DX ; citire si testare
RCR AL, 1
JNC PAR_WAIT
MOV AL, CL ; se preia caracterul din CL
MOV DX, 0D70H ; ADRESA PORT A (Exemplu S2=0)
OUT DX, AL
MOV DX, 0D72H ; ADRESA PORT C (Exemplu S2=0)
OR AL, 01H
OUT DX, AL
AND AL, 00H
OUT DX, AL  ; /STB = 0
OR AL, 01H
OUT DX, AL ; /STB = 1
RET
```

Rutina de scanare a minitastaturii:

```
SCAN_KEY:
   MOV BL, 0FEH      ; Masca initiala pentru primul rand (1111 1110)
   MOV BH, 0         ; Contor linie (0-3)
NEXT_ROW:
   MOV DX, 0D72H     ; Adresa PORT B (Linii/Rânduri)
   MOV AL, BL
   OUT DX, AL        ; Activeaza linia curenta (pune 0 pe bitul corespunzator)
```

```
    MOV DX, 0D74H      ; Adresa PORT C
    IN AL, DX          ; Citeste coloanele
    AND AL, 07H        ; Pastreaza doar primii 3 biti (Coloanele PC0-PC2)
    CMP AL, 07H        ; Verifica daca toate sunt 1 (nicio tasta apasata)
    JNE KEY_FOUND      ; Daca nu sunt toate 1, o tasta e apasata -> Sari la procesare

    ROL BL, 1          ; Muta zeroul la stanga pentru urmatoarea linie
    INC BH             ; Incrementeaza indexul liniei
    CMP BH, 4          ; Am verificat toate cele 4 linii?
    JB NEXT_ROW        ; Daca nu, repeta pentru urmatorul rand

    MOV AL, 0FFH       ; Daca nu s-a apasat nimic, returneaza FF
    RET

KEY_FOUND:
    ; Aici se poate adauga logica de conversie (Lookup Table)
    ; In acest moment: BH = Numarul Liniei, AL = Codul Coloanei
    ; Exemplu simplificat: Returnam in AH linia si in AL coloana
    MOV AH, BH
    RET
```

Rutina de aprindere/stingere a unui led:

```
; Intrare: AL = Masca de bit pentru LED-ul dorit (ex: 00000001 pentru LED1)
; AH = 1 (Aprinde) sau 0 (Stinge)
CONTROL_LED:
    PUSH BX            ; Salvam registrul BX
    MOV BL, [LED_STATE]; Citim starea curenta a LED-urilor din memorie (variabila
RAM)
```

```
    CMP AH, 1
    JE APRINDE

STINGE:
    NOT AL          ; Inversam masca pentru operatia AND
    AND BL, AL      ; Stingem bitul selectat (AND cu 0)
    JMP OUTPUT_LED

APRINDE:
    OR BL, AL       ; Aprindem bitul selectat (OR cu 1)

OUTPUT_LED:
    MOV [LED_STATE], BL ; Salvam noua stare in memorie
    MOV AL, BL
    MOV DX, 0800H
    OUT DX, AL      ; Trimite comanda catre LED-uri

    POP BX          ; Refacem BX
    RET

LED_STATE DB 00H    ; Variabila in segmentul de date
```

Rutina de afișare a unui caracter hexa pe un rang cu segmente:

```
; Intrare: AL = Cifra de afisat (0-F)
;        AH = Selectia Display-ului (0 = Display 1, 1 = Display 2)
DISPLAY_DIGIT:
    PUSH BX
    PUSH DX
    ; 1. Pregatirea codului 7-segmente
    LEA BX, TABLE_7SEG ; Incarca adresa tabelei de conversie
```

```
    XLAT            ; Converteste AL din hex in cod 7-segmente (AL <- [BX+AL])
    PUSH AX         ; Salvam codul segmentelor pe stiva temporar
    ; 2. Selectarea Display-ului (prin PC4)
    MOV DX, 0D74H   ; Adresa PORT C
    IN AL, DX       ; Citim starea curenta a portului C (pentru a nu strica tastatura)

    CMP AH, 1
    JE SELECT_DS2

SELECT_DS1:         ; Vrem Display 1 -> PC4 trebuie sa fie 0
    AND AL, 0EFH    ; Fortam bitul 4 la 0 (Mask: 1110 1111)
    JMP SET_SELECT

SELECT_DS2:         ; Vrem Display 2 -> PC4 trebuie sa fie 1
    OR AL, 10H      ; Fortam bitul 4 la 1 (Mask: 0001 0000)

SET_SELECT:
    OUT DX, AL      ; Trimitem comanda de selectie la 8255
    ; 3. Trimiterea Datelor pe Segmente (Port A)
    POP AX          ; Recuperam codul segmentelor din stiva (era in AL)
    MOV DX, 0D70H   ; Adresa PORT A
    OUT DX, AL      ; Trimitem codul cifrei catre segmente

    POP DX
    POP BX
    RET
; Tabela de conversie (Catod Comun: 1=Aprins)
; Ordinea segmentelor: DP g f e d c b a
TABLE_7SEG DB 3FH, 06H, 5BH, 4FH, 66H, 6DH, 7DH, 07H, 7FH, 6FH, 77H, 7CH, 39H,
5EH, 79H, 71H
;           0   1   2   3   4   5   6   7   8   9   A   B   C   D   E   F
```

BIBLIOGRAFIE

1. Brey, Barry B. The Intel Microprocessors: Architecture, Programming, and Interfacing (8086/8088, 80186/80188, 80286, 80386, 80486, Pentium, Pentium Pro Processor, Pentium II, Pentium III, Pentium 4, and Core2 with 64-bit Extensions). 8th Edition. Upper Saddle River, New Jersey: Pearson Education / Prentice Hall, 2009.

    - Referință: Capitolul 11 – "The 8255 Programmable Peripheral Interface", paginile 382-390.

2. Popa, Mircea; Gordan, Mihaela. Microprocesoare: Aplicații. Timișoara: Editura Politehnica, 2011. (Sau ediții mai noi utilizate la cursul de la UPT).

    - Referință: Secțiunile privind interfațarea memoriei și a perifericelor standard.

3. Intel Corporation. Microsystem Components Handbook, Volume 1 & 2. Santa Clara, California: Intel Corporation, 1986.

    - Referință: Specificațiile arhitecturale pentru familia iAPX 86.

4. Intel Corporation. 8255A/8255A-5 Programmable Peripheral Interface Datasheet.

    - Accesat: 09.12.2025, ora 14:15.

    - Referință: Paginile 1-6 (Pin Configuration și Block Diagram) și Paginile 12-14 (Mode 0 Operation).

    - Sursă: Arhiva tehnică Intel / Mouser Electronics.

5. Texas Instruments. SN74LS138 - 3-Line To 8-Line Decoders/Demultiplexers Datasheet. (Revised March 1988).

    - Accesat: 09.12.2025, ora 14:20.

    - Referință: Pagina 2 (Logic Diagram) și Pagina 3 (Function Table – intrările de Enable G1, G2).

6. Kingbright. Subject: SC56-11EWA - 14.22mm (0.56INCH) Single Digit Numeric Display Datasheet.

    - Accesat: 09.12.2025, ora 14:25.

    - Referință: Specificațiile pentru configurația "Common Cathode" și diagrama pinilor.

7. Intel Corporation. 8251A Programmable Communication Interface Datasheet.

    - Accesat: 09.12.2025, ora 14:30.

    - Referință: Secțiunea "Pin Description" (TxD, RxD, C/D).

8. All About Circuits. Microprocessor Interfacing: Memory and I/O.

   - Link: https://www.allaboutcircuits.com/textbook/digital/chpt-13/microprocessor-programming/

   - Accesat: 09.12.2025, ora 13:45.

   - Referință: Paragraful "Address Decoding" și interfațarea memoriilor EPROM/SRAM.

9. GeeksforGeeks. 8255 Architecture and Operating Modes.

   - Link: https://www.geeksforgeeks.org/programmable-peripheral-interface-8255/

   - Accesat: 09.12.2025, ora 13:50.

   - Referință: Secțiunea "Control Word Register" și "Mode 0: Simple Input/Output".